



University
of Exeter

COMM113

Deep Learning – Week 11

Trustworthy Deep Learning

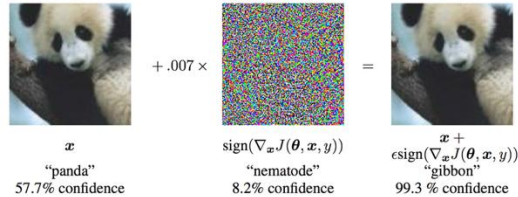
Gaojie (Jay) Jin
g.jin@exeter.ac.uk

Why Trustworthy Deep Learning?

- High-stakes deployment: healthcare, finance, autonomous driving.



- Case studies of failures (adversarial examples in vision; GPT hallucinations).



- Regulatory landscape: EU AI Act, UK AI Safety Summit.



Overview

- **Robustness**
- **Generalisation**
- **Fairness**
- **Interpretability**

Overview

- **Robustness**
- **Generalisation**
- **Fairness**
- **Interpretability**

Q: Why might robustness and generalisation be related but not identical?

Overview

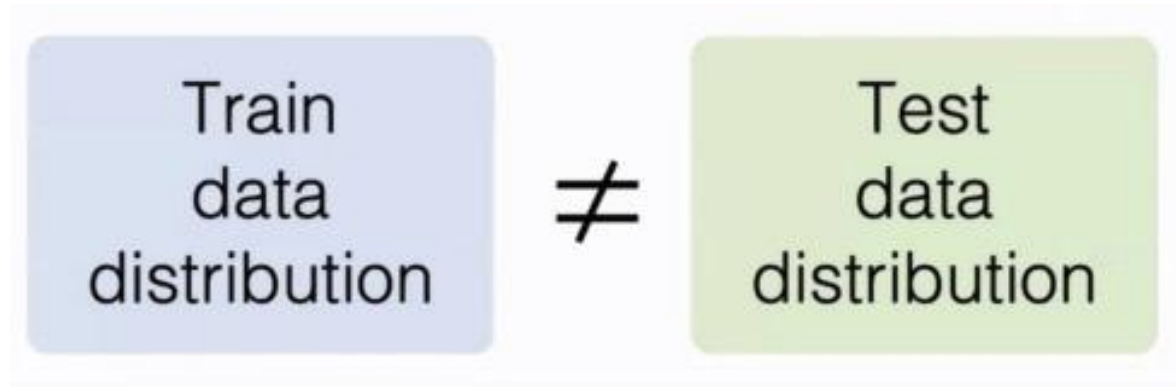
- **Robustness**
- **Generalisation**
- **Fairness**
- **Interpretability**

Q: Why might robustness and generalisation be related but not identical?

A: Generalisation handles natural variations; robustness handles worst-case or shifted variations.

Robustness – Distribution Shift (Out-of-Distribution Robustness)

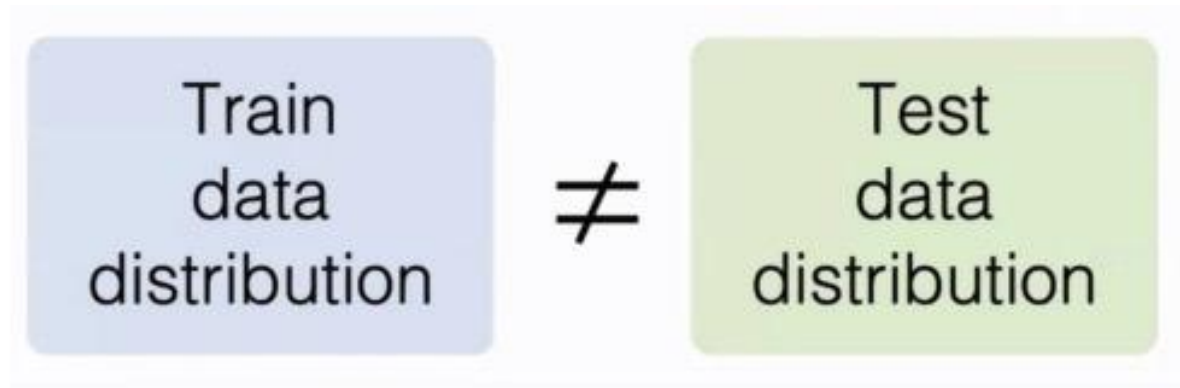
Distribution shift occurs when training and test inputs follow different distributions. Even small changes in environment or context can cause failure.



Q: Is distribution shift always adversarial?

Robustness – Distribution Shift (Out-of-Distribution Robustness)

Distribution shift occurs when training and test inputs follow different distributions. Even small changes in environment or context can cause failure.



Q: Is distribution shift always adversarial?

A: No — it can be natural (e.g., lighting, camera changes).

Robustness – Distribution Shift

- Example: Dynamic environment



Source: MNIST



Target: SVHN

Q: Why does an MNIST-trained CNN fail on SVHN even though the labels are the same?

Robustness – Distribution Shift

- Example: Dynamic environment



Source: MNIST



Target: SVHN

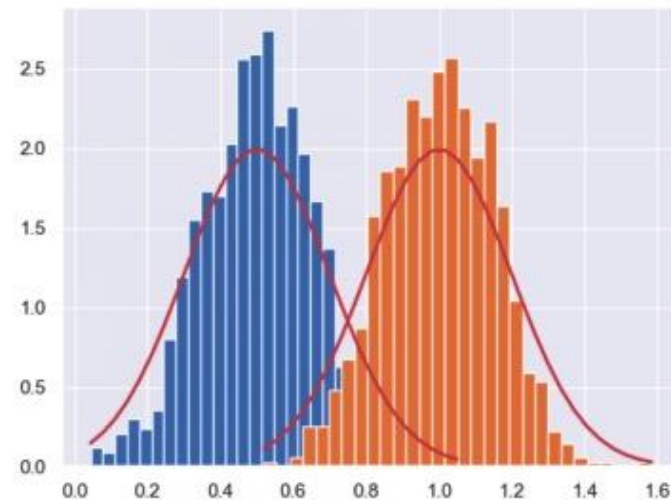
Q: Why does an MNIST-trained CNN fail on SVHN even though the labels are the same?

A: Because the visual distribution (colour, background, texture) differs significantly, and the model has not learned invariant features.

Robustness – Distribution Shift

Training images and test images are from different distributions:

- Different resolution (high res. SLR vs. low res. surveillance)
- Different scale, perspective and pose (closeup of face vs. full body)
- Different lighting and colours (indoors colour vs. outdoor IR/BW)
- Different label distribution (other cats in image, multiple cats in image)

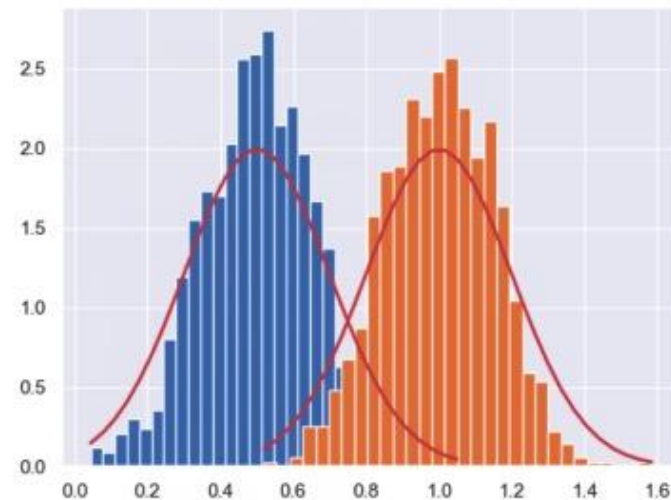


Q: Which cause of distribution shift is hardest to model during training?

Robustness – Distribution Shift

Training images and test images are from different distributions:

- Different resolution (high res. SLR vs. low res. surveillance)
- Different scale, perspective and pose (closeup of face vs. full body)
- Different lighting and colours (indoors colour vs. outdoor IR/BW)
- Different label distribution (other cats in image, multiple cats in image)

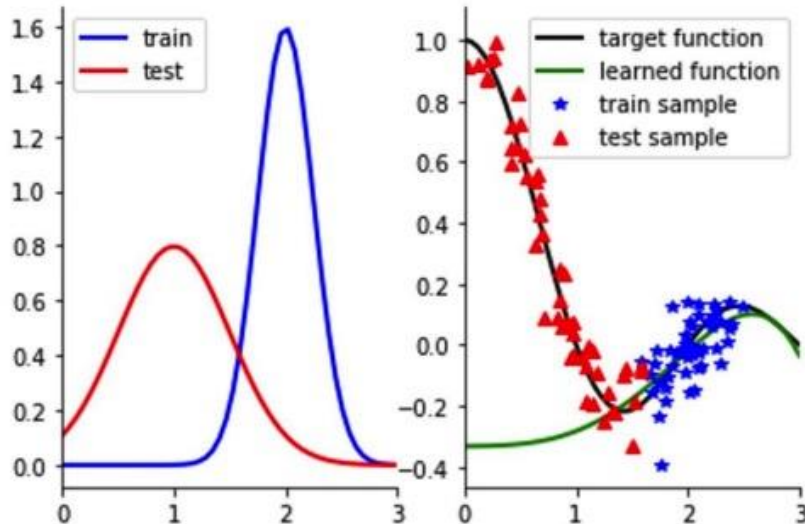


Q: Which cause of distribution shift is hardest to model during training?

A: Label distribution shift or causal shift — because it may require changes in model semantics rather than just input augmentation.

Robustness – Distribution Shift

- Distribution shifts can cause models to fail
- Training and test input follow different distributions
- Underlying target function remains unchanged



Distribution shift causes failures even if training accuracy is high. Because training and test have different distributions but the underlying relationship $f(x)$ is unchanged.

Learning under Distribution Shift

- Empirical Risk Minimization (ERM)

Let $f_{\mathbf{w}} : \mathcal{X} \rightarrow \mathcal{Y}$ denote a predictor **parameterized by $\mathbf{w} \in \mathbb{R}^d$** . ERM:

$$\min_{\mathbf{w}} \left\{ R_n(\mathbf{w}) := \frac{1}{n} \sum_{i=1}^n \ell(f_{\mathbf{w}}(\mathbf{x}_i), \mathbf{y}_i) \right\}$$

where $\ell(f_{\mathbf{w}}(\mathbf{x}_i), \mathbf{y}_i)$ is the loss on $(\mathbf{x}_i, \mathbf{y}_i) \stackrel{\text{i.i.d.}}{\sim} p^{\text{tr}}(\mathbf{x}, \mathbf{y})$

ERM is over $p^{\text{tr}}(\mathbf{x}, \mathbf{y})$

ERM minimises average loss over the training distribution. But ...

Q: Why is ERM insufficient under distribution shift?

$f_{\mathbf{w}^*}$ is not guaranteed to generalize well on $p^{\text{te}}(\mathbf{x}, \mathbf{y})$

Learning under Distribution Shift

- Empirical Risk Minimization (ERM)

Let $f_{\mathbf{w}} : \mathcal{X} \rightarrow \mathcal{Y}$ denote a predictor **parameterized by $\mathbf{w} \in \mathbb{R}^d$** . ERM:

$$\min_{\mathbf{w}} \left\{ R_n(\mathbf{w}) := \frac{1}{n} \sum_{i=1}^n \ell(f_{\mathbf{w}}(\mathbf{x}_i), \mathbf{y}_i) \right\}$$

where $\ell(f_{\mathbf{w}}(\mathbf{x}_i), \mathbf{y}_i)$ is the loss on $(\mathbf{x}_i, \mathbf{y}_i) \stackrel{\text{i.i.d.}}{\sim} p^{\text{tr}}(\mathbf{x}, \mathbf{y})$

ERM is over $p^{\text{tr}}(\mathbf{x}, \mathbf{y})$

ERM minimises average loss over the training distribution. But ...

Q: Why is ERM insufficient under distribution shift?

A: Because it optimises only for the training distribution, not for unseen or shifted distributions.

$f_{\mathbf{w}^*}$ is not guaranteed to generalize well on $p^{\text{te}}(\mathbf{x}, \mathbf{y})$

Learning under Distribution Shift

- Distributionally Robust Optimisation (DRO)

Let $f_{\mathbf{w}} : \mathcal{X} \rightarrow \mathcal{Y}$ denote a predictor **parameterized by $\mathbf{w} \in \mathbb{R}^d$** . DRO:

$$\min_{\mathbf{w}} \left\{ R(\mathbf{w}) := \sup_{Q \in \mathcal{Q}} \hat{\mathbb{E}}_{(\mathbf{x}, \mathbf{y}) \sim Q} [\ell(f_{\mathbf{w}}(\mathbf{x}), \mathbf{y})] \right\}$$

Minimise the worst-case loss over an uncertainty set of distributions

DRO can lead to overly pessimistic models focusing on unrealistic worst-case distributions

Q: What is the main drawback of DRO?

Learning under Distribution Shift

- Distributionally Robust Optimisation (DRO)

Let $f_{\mathbf{w}} : \mathcal{X} \rightarrow \mathcal{Y}$ denote a predictor **parameterized by $\mathbf{w} \in \mathbb{R}^d$** . DRO:

$$\min_{\mathbf{w}} \left\{ R(\mathbf{w}) := \sup_{Q \in \mathcal{Q}} \hat{\mathbb{E}}_{(\mathbf{x}, \mathbf{y}) \sim Q} [\ell(f_{\mathbf{w}}(\mathbf{x}), \mathbf{y})] \right\}$$

Minimise the worst-case loss over an uncertainty set of distributions

DRO can lead to overly pessimistic models focusing on unrealistic worst-case distributions

Q: What is the main drawback of DRO?

A: It may reduce accuracy by overfitting to extreme worst-case distributions.

Learning under Distribution Shift

- Invariant Risk Minimization (IRM)

$$\begin{aligned} & \min_{\substack{\Phi: \mathcal{X} \rightarrow \mathcal{H} \\ w: \mathcal{H} \rightarrow \mathcal{Y}}} \sum_{e \in \mathcal{E}_{\text{tr}}} R^e(w \circ \Phi) \\ & \text{subject to } w \in \arg \min_{\bar{w}: \mathcal{H} \rightarrow \mathcal{Y}} R^e(\bar{w} \circ \Phi), \text{ for all } e \in \mathcal{E}_{\text{tr}}. \end{aligned} \tag{IRM}$$

Train models to rely on features that are consistent across different data environments, rather than spurious correlations that may only exist in the training data.

Q: Why does IRM require multiple environments?

Learning under Distribution Shift

- Invariant Risk Minimization (IRM)

$$\begin{aligned} & \min_{\substack{\Phi: \mathcal{X} \rightarrow \mathcal{H} \\ w: \mathcal{H} \rightarrow \mathcal{Y}}} \sum_{e \in \mathcal{E}_{\text{tr}}} R^e(w \circ \Phi) \\ & \text{subject to } w \in \arg \min_{\bar{w}: \mathcal{H} \rightarrow \mathcal{Y}} R^e(\bar{w} \circ \Phi), \text{ for all } e \in \mathcal{E}_{\text{tr}}. \end{aligned} \tag{IRM}$$

Train models to rely on features that are consistent across different data environments, rather than spurious correlations that may only exist in the training data.

Q: Why does IRM require multiple environments?

A: Because spurious correlations differ across environments, helping the model discover invariant features.

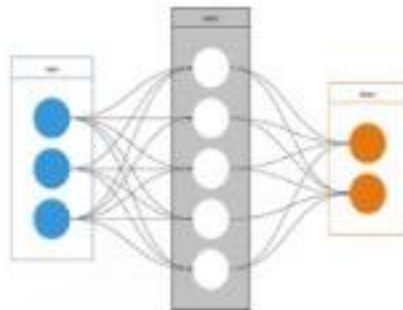
Adversarial Robustness

- A motivation example

Given a well-trained classification model for ImageNet. How to **alter the model output by changing the input?**



Input image x



$f(\cdot)$ (fixed)

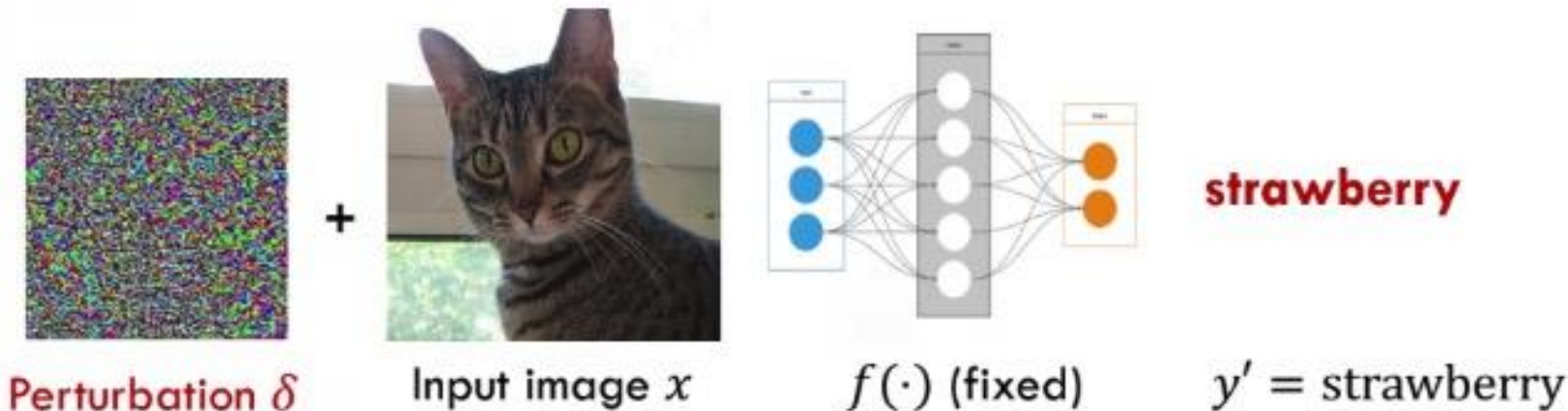
“tabby cat”

$y = \text{tabby cat}$

Adversarial Robustness

- A motivation example

Given a well-trained classification model for ImageNet. How to **alter the model output by changing the input?**



1. Rephrase what we want:

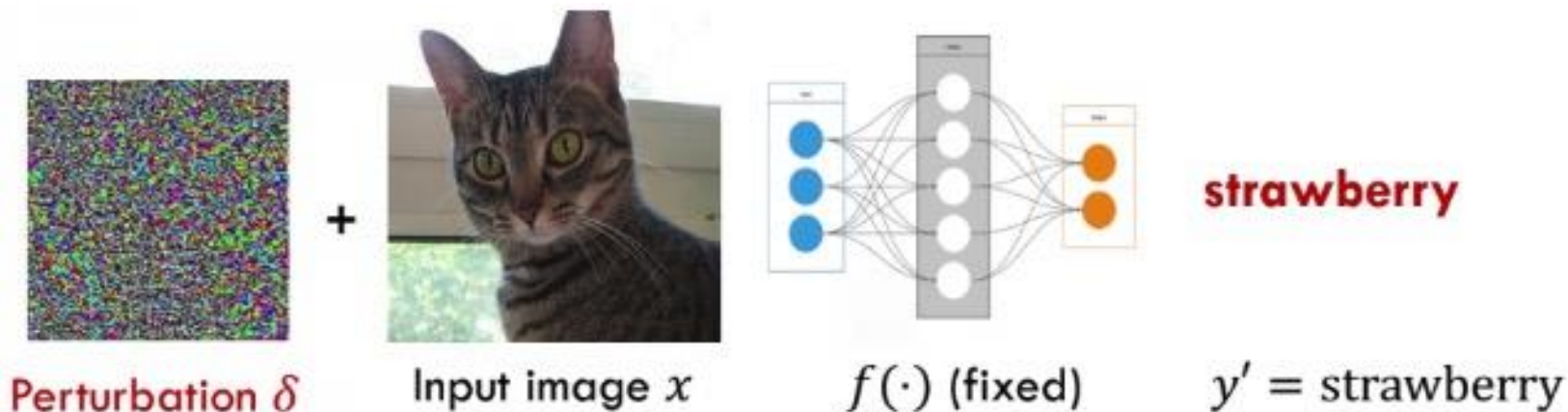
Find δ such that $f(x + \delta) = y' \neq y$

For example, $y' = \text{strawberry}$

Adversarial Robustness

- A motivation example

Given a well-trained classification model for ImageNet. How to **alter the model output by changing the input?**



1. Rephrase what we want:

Find δ such that $f(x + \delta) = y' \neq y$
For example, $y' = \text{strawberry}$

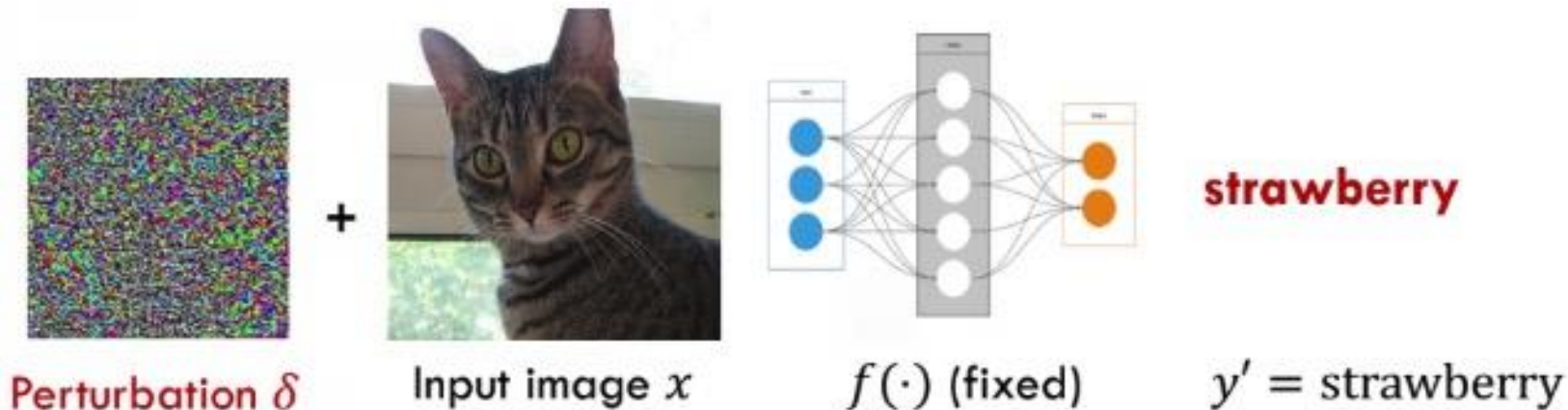
2. Defining the loss function:

$$\min_{\delta} L(f(x + \delta), y')$$

Adversarial Robustness

- A motivation example

Given a well-trained classification model for ImageNet. How to **alter the model output by changing the input?**



1. Rephrase what we want

Find δ such that $f(x + \delta) = y' \neq y$
For example, $y' = \text{strawberry}$

2. Defining the loss function

$$\min_{\delta} L(f(x + \delta), y')$$

3. Optimise the perturbation

$$\delta = \delta - \alpha \cdot \frac{\partial L(\delta, y')}{\partial \delta}$$

Adversarial Attack

- **Fast Gradient Sign Method (FGSM) attack**

Given:

- input x
- true label y
- model parameters θ
- loss function $\mathcal{L}(\theta, x, y)$
- perturbation size ϵ
- FGSM computes the gradient once and takes a single step in the sign direction.
- The sign ensures maximum change under an L^∞ or L_2 budget.
- This is fast and effective but usually weaker than iterative attacks.

The adversarial example is:

$$x_{\text{adv}} = x + \epsilon \cdot \text{sign}(\nabla_x \mathcal{L}(\theta, x, y))$$

Take the sign of the gradient, not the raw gradient → ensures a max-norm bounded perturbation.

Adversarial Attack

- PGD attack (Projected Gradient Descent attack)

PGD is essentially an **iterative version** of FGSM.

Given:

- Input x
- True label y
- Loss $\mathcal{L}(\theta, x, y)$
- Step size α
- Number of iterations T
- Allowed perturbation size ϵ

Start from either a random point within the ball:

$$x^{(0)} = x + \text{Uniform}[-\epsilon, \epsilon]$$

Iterate:

$$x^{(t+1)} = x^{(t)} + \alpha \cdot \text{sign}(\nabla_{x^{(t)}} \mathcal{L}(\theta, x^{(t)}, y))$$

Project back to the ℓ_∞ -ball:

$$x^{(t+1)} = \Pi_{B_\epsilon(x)}(x^{(t+1)})$$

PGD (especially PGD-50 or PGD-100) is used to evaluate adversarial robustness in almost all robustness papers.

Also clamp to valid pixel range:

$$x^{(t+1)} = \text{clip}(x^{(t+1)}, 0, 1)$$

Final adversarial example: $x_{\text{adv}} = x^{(T)}$.

Adversarial Attack

- **PGD attack (Projected Gradient Descent attack)**

PGD is essentially an **iterative version** of FGSM.

Given:

- Input x
- True label y
- Loss $\mathcal{L}(\theta, x, y)$
- Step size α
- Number of iterations T
- Allowed perturbation size ϵ

Q: Why does PGD outperform FGSM?

PGD (especially PGD-50 or PGD-100) is used to evaluate adversarial robustness in almost all robustness papers.

Adversarial Attack

- **PGD attack (Projected Gradient Descent attack)**

PGD is essentially an **iterative version** of FGSM.

Given:

- Input x
- True label y
- Loss $\mathcal{L}(\theta, x, y)$
- Step size α
- Number of iterations T
- Allowed perturbation size ϵ

Q: Why does PGD outperform FGSM?

A: Because multiple small steps allow more precise optimisation of the adversarial objective.

PGD (especially PGD-50 or PGD-100) is used to evaluate adversarial robustness in almost all robustness papers.

How to improve Adversarial Robustness?

- **Adversarial Training**

Standard training minimises average loss:

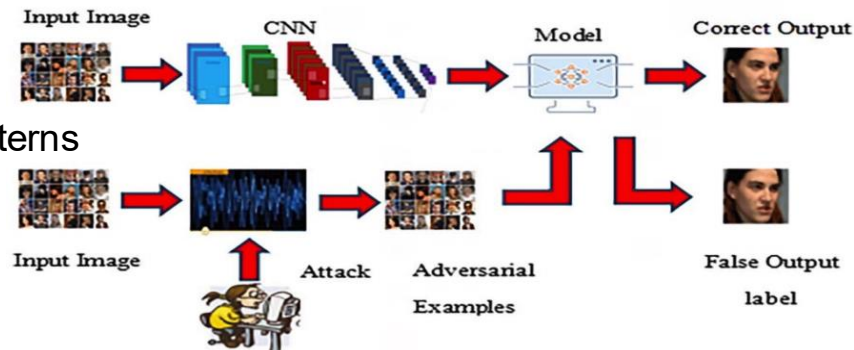
$$\min_{\theta} \mathbb{E}_{(x,y)} [\mathcal{L}(\theta, x, y)]$$

Adversarial training instead minimises the worst-case loss inside an allowed perturbation region:

$$\min_{\theta} \mathbb{E}_{(x,y)} \left[\max_{\|\delta\| \leq \epsilon} \mathcal{L}(\theta, x + \delta, y) \right]$$

Why Adversarial Training Works

- Forces model to rely on robust features, not fragile patterns
- Sharpens the decision boundary
- Reduces model's local sensitivity to perturbations



How to improve Adversarial Robustness?

- Adversarial Training

But ...

Standard training minimises average loss:

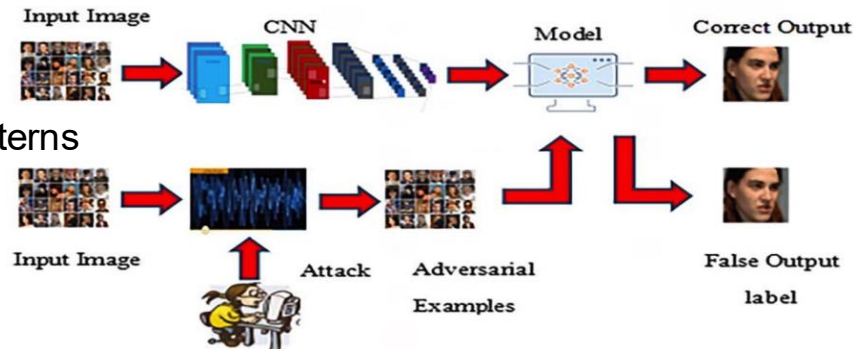
$$\min_{\theta} \mathbb{E}_{(x,y)} [\mathcal{L}(\theta, x, y)]$$

Adversarial training instead minimises the worst-case loss inside an allowed perturbation region:

$$\min_{\theta} \mathbb{E}_{(x,y)} \left[\max_{\|\delta\| \leq \epsilon} \mathcal{L}(\theta, x + \delta, y) \right]$$

Why Adversarial Training Works

- Forces model to rely on robust features, not fragile patterns
- Sharpens the decision boundary
- Reduces model's local sensitivity to perturbations



How to improve Adversarial Robustness?

- Adversarial Training

Standard training minimises average loss:

$$\min_{\theta} \mathbb{E}_{(x,y)} [\mathcal{L}(\theta, x, y)]$$

Adversarial training instead minimises the worst-case loss inside an allowed perturbation region:

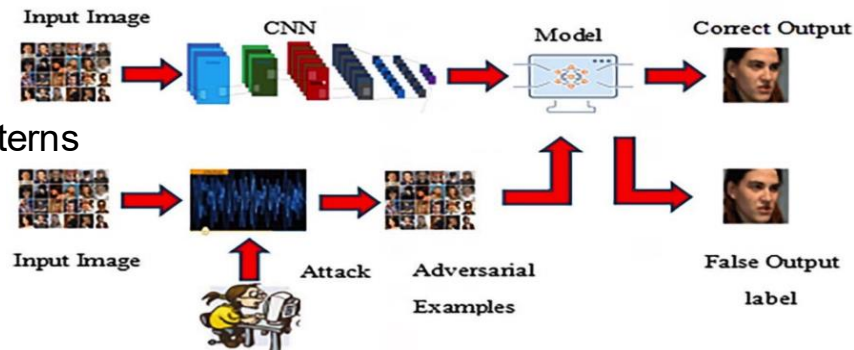
$$\min_{\theta} \mathbb{E}_{(x,y)} \left[\max_{\|\delta\| \leq \epsilon} \mathcal{L}(\theta, x + \delta, y) \right]$$

Why Adversarial Training Works

- Forces model to rely on robust features, not fragile patterns
- Sharpens the decision boundary
- Reduces model's local sensitivity to perturbations

But ...

Training time.



How to improve Adversarial Robustness?

- **Adversarial Training - TRADES**

TRADES is a refined and theoretically grounded form of adversarial training that explicitly balances clean accuracy and robust accuracy.

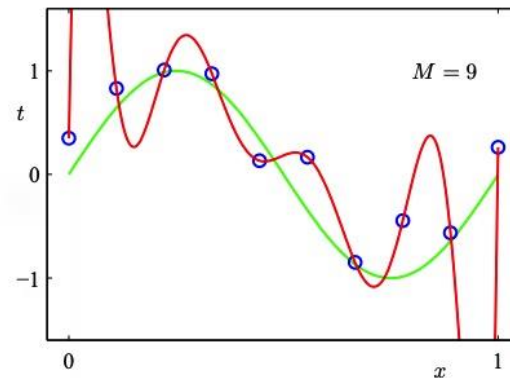
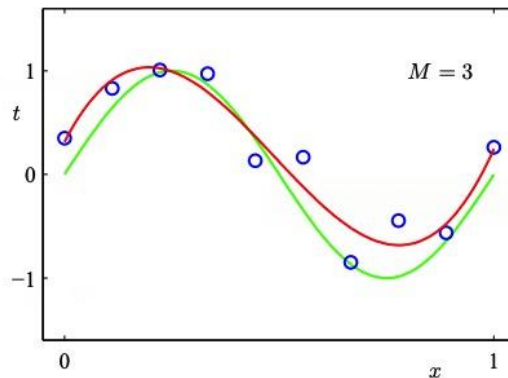
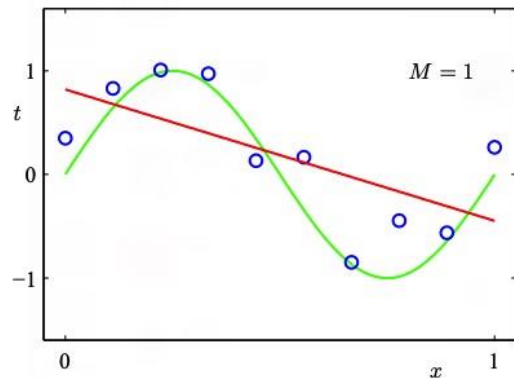
$$\min_{\theta} \mathbb{E}_{(x,y)} \left[\underbrace{\mathcal{L}(f_{\theta}(x), y)}_{\text{clean loss}} + \underbrace{\beta \cdot \text{KL}(f_{\theta}(x) \parallel f_{\theta}(x_{\text{adv}}))}_{\text{robustness term}} \right]$$

Why TRADES Is Better Than Standard Adversarial Training

- Explicit control of accuracy–robustness trade-off
- Better clean accuracy at comparable robustness
- Theoretically grounded: derived from a tight surrogate upper bound on the robust classification error.
- More stable inner maximisation: KL gives smoother gradients than cross-entropy.

Generalisation

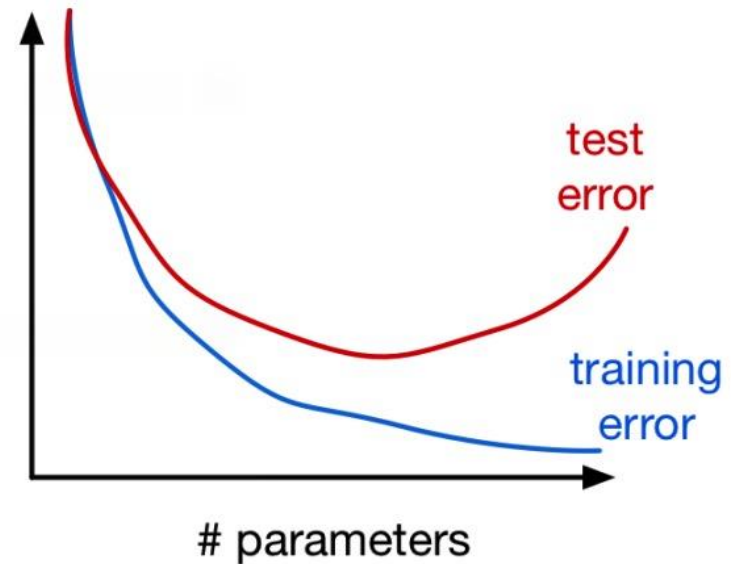
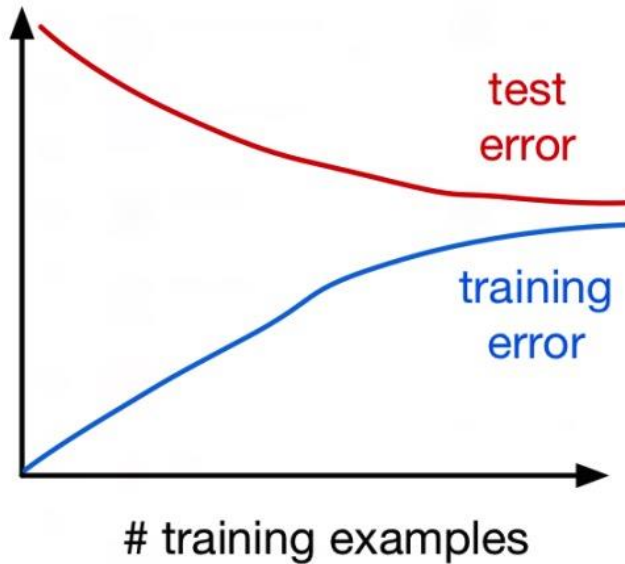
- Recall: overfitting and underfitting



We'd like to minimise the generalisation error, i.e. error on new examples.

Generalisation

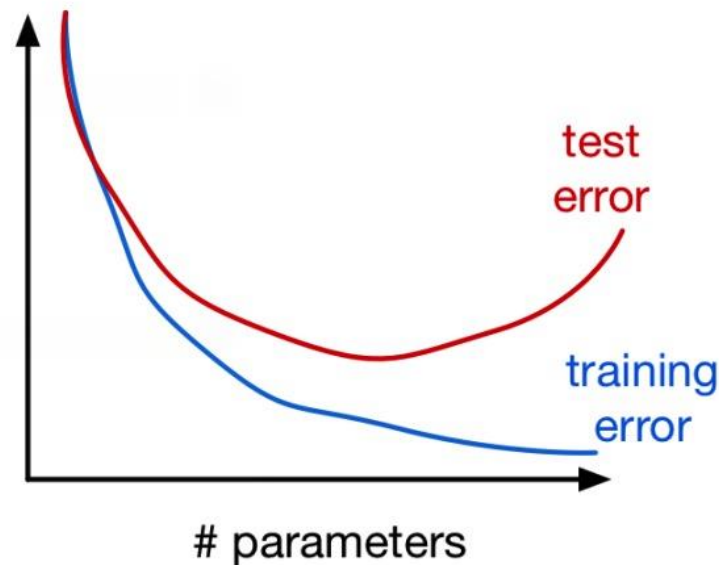
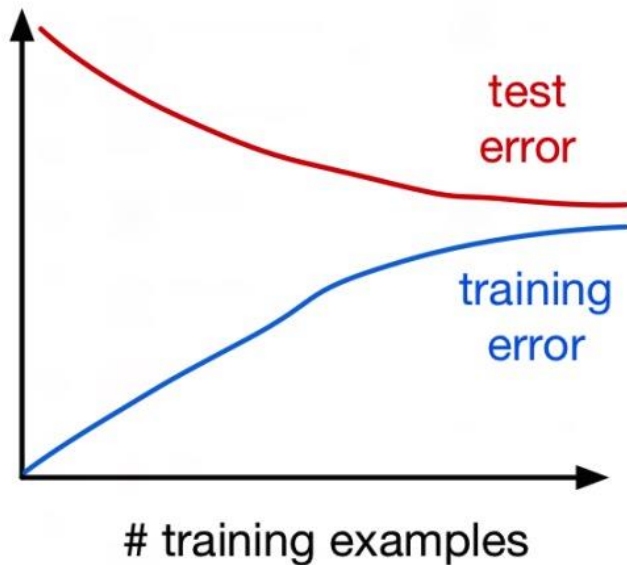
- Training and test error as a function of # training examples and # parameters



Q: Why does increasing complexity reduce training error but may increase test error?

Generalisation

- Training and test error as a function of # training examples and # parameters



Q: Why does increasing complexity reduce training error but may increase test error?

A: Because highly flexible models can memorise noise rather than learn general patterns.

Generalisation

How can we train a model that's complex enough to model the structure in the data, but prevent it from overfitting?

Our bag of tricks

- data augmentation
- reduce the number of parameters
- weight decay
- early stopping
- label smoothing
- stochastic regularization (e.g. dropout)

The best-performing models on most benchmarks use some or all of these tricks.

How to bound the generalisation?

What Problem Are We Solving?

When we train a machine learning model, we want it to:

- perform well on the training data, and also
- perform well on unseen test data.

This is the generalisation problem.

How to bound it?

- **PAC-Bayes**
- **VC-Dimension**

Q: Why do we need generalisation bounds at all?

How to bound the generalisation?

What Problem Are We Solving?

When we train a machine learning model, we want it to:

- perform well on the training data, and also
- perform well on unseen test data.

This is the generalisation problem.

How to bound it?

- **PAC-Bayes**
- **VC-Dimension**

Q: Why do we need generalisation bounds at all?

A: To reason formally about model reliability beyond empirical accuracy.

How to bound the generalisation?

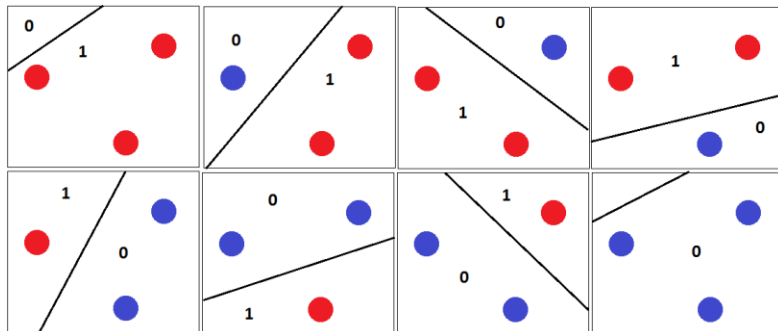
- **VC-dimension**

VC-dimension (Vapnik–Chervonenkis dimension) measures:

How many points the model class can label in all possible ways.

- If a model class can fit any labelling of h points (no matter how random), we say it shatters those points.

The VC-dimension is the largest h for which this is possible.



Imagine trying to classify points in 2D using a straight line.

Can it shatter 1 point? → Yes

2 points? → Yes

3 arbitrary points? → Yes

4 arbitrary points? → No

Some labelings cannot be separated by a straight line.

So: VC-dim (linear classifier in 2D)=3

How to bound the generalisation?

- **VC-dimension**

A famous result in learning theory says:

- If the VC-dimension is small relative to the number of training samples, then your model will generalise well.

$$\text{Test error} \approx \text{Training error} + O\left(\sqrt{\frac{\text{VC-dim}}{n}}\right)$$

Where n = number of training samples.

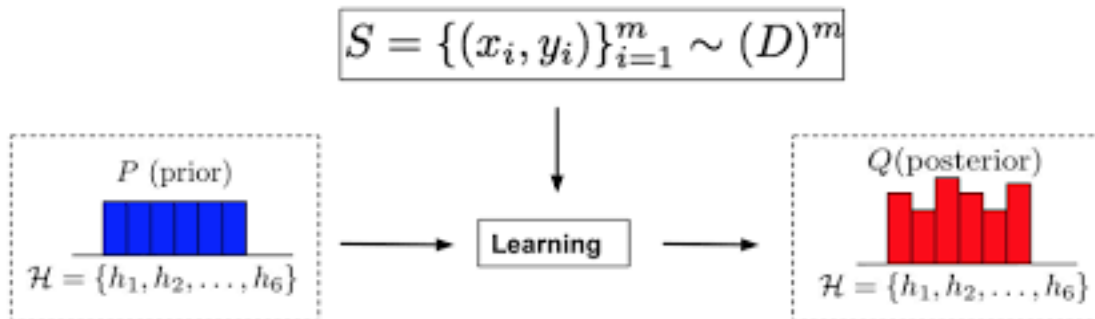
In DNNs, their VC-dimension is roughly proportional to the number of parameters, often millions.

How to bound the generalisation?

- **PAC-Bayes**

Classical tools (like VC dimension) struggle to analyse modern deep neural networks, they have millions of parameters and can even memorise random labels.

PAC-Bayes is a modern theoretical tool that helps us understand generalisation even for large, over-parameterised models.



How to bound the generalisation?

- **PAC-Bayes**

Think of it this way:

- Before training, we have some belief (a prior) about what good models might look like.
- After training, we update our belief to a new distribution (a posterior) that prefers models that fit the data well.
- The PAC-Bayes theorem connects:
 1. how well models in the posterior do on training data, and
 2. how well they will do on new data.

It gives a guarantee that if the posterior stays “close” to the prior, then good performance on the training data means good performance on unseen data.

How to bound the generalisation?

- **PAC-Bayes**

Prior P

Your belief *before* seeing the data.

Example: "Weights near zero are more likely."

Posterior Q

Your belief *after* training.

Example: "Weights near these trained values are more likely."

PAC-Bayes tells us:

If Q is not too far from P , then good training error implies good generalisation.

The "distance" is measured by **KL divergence** (just think of it as a number telling us how different Q is from P).

How to bound the generalisation?

- PAC-Bayes

$$\mathbb{E}_{\Theta \sim Q}[\mathcal{L}_D(f_\Theta)] \leq \mathbb{E}_{\Theta \sim Q}[\mathcal{L}_S(f_\Theta)] + \sqrt{\frac{\text{KL}(Q||P) + \log \frac{m}{\delta}}{2(m-1)}},$$

Test error $\leq$ (Training error) + (how far the posterior moved from the prior) / $\sqrt{\text{(number of training samples)}}$

This captures an important intuition:

- If the model fits the data and
- If training does not make the posterior distribution too different from the prior

The model will generalise well.

How to bound the generalisation?

- **PAC-Bayes (Example Intuition)**

Suppose you start with the prior belief:

- “A good model probably has small weights.”

Now you train a neural network. After training, if the weights are indeed small (close to your prior belief), PAC-Bayes says:

- Since the posterior is near the prior AND the training error is low, your generalisation error is guaranteed to be low.

If training drastically changes the weights (posterior far from prior), the bound says:

- Be careful! The model might overfit.

This matches real deep learning behaviour!

How to bound the generalisation?

- **PAC-Bayes and VC-dimension**

VC-dimension is a worst-case, model-class-wide measure
→ often too loose for analysing deep networks.

PAC-Bayes is data-dependent and posterior-dependent
→ gives much tighter generalisation guarantees for real neural networks.

Next: Fairness and Interpretability